Production Handover Guide

Neighbourhood Livability Index

Prepared 19 October 2026 | Repository: prasad830105/DEMOGRAFY_INT | Main branch commit: 4229b1d

Purpose and release status

This guide gives the product and integration teams the current application behavior, data contract, design decisions, and production work still required. The repository is a working Streamlit prototype. It is not yet a production release: deployment, security hardening, measured performance, and iframe validation remain open.

The current scope is a two-suburb comparison of mapped parks and related green or recreation features. It uses OpenStreetMap through public Overpass endpoints. Google Places is not implemented, so Google-specific request limits and cost controls are not active in this version.

Current delivery snapshot

Area	Current implementation	Handover status
User experience	Search for two Australian suburbs, compare, see counts, area, density, pair-relative scores and a bar chart.	Implemented
Data source	Overpass query for leisure=park, garden, nature_reserve and playground within CSV latitude/longitude rectangles.	Implemented with OSM; confirm as approved source
Persistence	Supabase suburb catalogue and amenity_counts; successful live counts are upserted.	Prototype configuration; harden before production
Transformations	Density and pair score are calculated in app.py.	No dbt or SQL model
Delivery	Repository is on GitHub; local Streamlit app currently runs at localhost:8501.	Production deployment and iframe not verified

Source of truth

The latest implementation is in the main branch of https://github.com/prasad830105/DEMOGRAFY_INT. This handover describes commit 4229b1d (Remove walkable network score). The walkable network feature and OpenRouteService calls have been removed.

System flow

The Streamlit process handles the comparison request, calls Supabase and Overpass as needed, computes the selected KPI, and renders the results. Supabase is both the suburb search store and the durable count cache.

Step	Current behavior	System
1. Select	Search and choose two different suburbs. Supabase is searched first; bundled suburbs_import.csv is the fallback.	Streamlit + Supabase REST
2. Resolve cache	Use Streamlit's 24-hour result cache. On a cache miss, look up saved OSM counts in Supabase.	Streamlit cache + PostgREST
3. Fetch	If either selected count is missing or a live refresh is requested, query public Overpass endpoints for those suburb rectangles.	Overpass API
4. Count and save	Deduplicate OSM element type and ID, count tagged features in each rectangle, then upsert the two counts to amenity_counts.	Python + Supabase REST
5. Transform and show	Divide count by source SAL area_km2, calculate the pair-relative score, and show a table, metrics and bar chart.	Python + Streamlit

KPI definition

Feature count is the number of unique OSM elements with leisure=park, leisure=garden, leisure=nature_reserve or leisure=playground whose representative point is inside the selected CSV rectangle. Density equals feature count divided by the suburb's supplied area_km2. The higher of the two densities is the pair benchmark. Score = min(100, 100 x suburb density / pair benchmark density); both scores are zero when both densities are zero.

The rectangle is an approximation of the suburb extent, not the SAL polygon. The denominator is the SAL area from the CSV. Counts can include features outside irregular, coastal or otherwise non-rectangular suburbs, which can bias density. Separate park, garden and playground records may refer to related physical spaces.

Data contract and cache behavior

Table	Role	Key fields
public.suburbs	Searchable active SAL catalogue used by the app.	suburb (key), sal_code, sal_name, state_abbr, legacy_name, area_km2, min_lat, min_lon, max_lat, max_lon, is_active
public.suburbs_import	CSV staging table used during catalogue refresh.	sal_code, suburb identity, area, bounds, representative centre and source fields
public.amenity_counts	One current count per suburb and category; also the durable cache.	suburb, category, place_count, source, updated_at; primary key (suburb, category)

Cache order and freshness

- Streamlit caches OSM count results for 24 hours in the running app process. This cache is not shared across independent app instances and is lost on restart or redeploy.
- On a process cache miss, the app reads saved rows from Supabase. Successful live results are upserted to Supabase so later app processes can reuse them.
- Supabase rows have updated_at, but the app does not enforce a maximum age when reading them. Production must agree a freshness policy and make stale data trigger a refresh or an explicit stale label.
- The Refresh live OSM counts action bypasses cached and saved counts and re-queries Overpass for the selected pair.

External interfaces

Interface	Current use	Operational note
Overpass API	POST Overpass QL queries for OSM leisure features in selected rectangles.	Public community endpoints have no app SLA; timeouts, throttling and outages are possible.
Supabase REST	Read active suburbs; read and upsert amenity_counts.	Uses SUPABASE_URL and SUPABASE_KEY on the Streamlit server.
Google Places	Not called by this release.	No Google result cap tracking or Google cost is active.

Production decisions to confirm

Decision	Current position	Production action
Approved data source	OSM replaces the Google Places source in the brief.	Product owner must confirm OSM as the accepted source and document coverage expectations.
Suburb boundary	CSV min/max coordinates form a rectangle.	Choose whether to accept the estimate or use official SAL polygon geometry for production counts.
Freshness	24-hour in-process cache; Supabase count rows have no read-time expiry.	Set an explicit TTL, last-updated label, and refresh policy.
KPI meaning	Feature density and a score relative only to the chosen pair.	Decide whether users need stable benchmarks, per-capita access, feature type weights or only raw counts/density.
Data retention	Only aggregate counts are stored; raw OSM response/history is not retained.	Confirm if audit history or raw feature IDs are required and check OSM attribution/licensing obligations.
Performance	No production latency measurement or under-15-second guarantee.	Instrument cache, DB, Overpass, transform and render stages; define p50/p95 target.
Embedding	No deployed production URL or iframe test is recorded.	Confirm host, access model and iframe/browser security requirements.

Security and release risks

The repository SQL policies are labeled for public illustrative/demo data. The current design uses the Supabase anon key from the Streamlit server and has policies that permit anonymous reads and writes for allowlisted demo and OSM source values. Do not carry those policies into a production project without a security review. Use least-privilege RLS and a trusted server-side write path; never put a service-role key in browser code or a committed file.

- Keep .env and Streamlit secrets out of Git. The repository includes .env.example only; provision real values through the deployment platform's secret store.
- Public Overpass endpoints can fail or rate-limit. Add bounded retries, timeouts, monitoring and a clear stale-data fallback before relying on this for customers.
- The current score does not measure actual walking access, park area, quality, entrances, safety or amenity availability. Label it as mapped-feature density.
- The app currently has no map, no Google Places integration, no dbt transformation, no raw feature history, and no recorded iframe deployment test.

Production readiness gate

The integration team can treat the following items as release gates. A deployment should remain in staging until security, data definitions and reliability checks are complete.

[ ] Product signs off on OSM as the source and approves the Parks feature definition, bounding method, area denominator and score wording.

[ ] Engineering chooses polygon boundaries or explicitly accepts the rectangular extent approximation and its effect on density.

[ ] Security reviews RLS, key handling, anonymous access, write path, abuse controls and data retention. Demo policies are not reused unchanged.

[ ] Freshness behavior is defined for the Supabase durable cache. The UI shows source and last-updated time for both current and fallback values.

[ ] Automated checks cover tag filtering, element deduplication, rectangle edge handling, density math, zero-count pairs and missing area values.

[ ] Integration checks cover cache hit, Supabase hit, incomplete cache, live Overpass success, Overpass failure, Supabase write failure and manual refresh.

[ ] Response-time telemetry captures cache lookup, Supabase, Overpass, transform and full request; representative cold and warm traffic meets the approved target.

[ ] Staging is deployed from the selected GitHub commit; secrets are configured outside Git; health checks and rollback steps are documented.

[ ] Iframe embedding is tested from the target host in supported browsers, including authentication, mobile layout and cross-site cookie behavior if applicable.

Handover package locations

- Application entry point: app.py
- KPI definition and caveats: KPI-card.md
- Setup and data behavior: README.md
- Supabase SQL: supabase_schema.sql, supabase_suburbs.sql, supabase_suburb_bounds.sql, supabase_suburb_bounds_sync.sql, supabase_osm_policy.sql and related migration scripts.